



湖南大学
HUNAN UNIVERSITY

回溯与分支限界

—— 湖南大学计算机学院 ——

小班讨论



分成六组:

第1-2组, 《算法导论》第16章思考题16-1、第23章思考题23-3

第3-4组, 《算法导论》第16章思考题16-2、第23章思考题23-1

第5-6组, 《算法导论》第22章思考题22-1、第22章思考题22-3

期末考试



- 一、判断题 (18分)
 - 二、计算题 (28分) 书本与课件上交集里面的经典问题求解
 - 三、算法题1 (15分)
 - 四、算法题2 (15分)
 - 五、算法题3 (12分) 书本与课件上交集里面的经典问题的算法分析与设计
 - 六、算法题4 (12分) 教材每章最后的思考题, 范围贪心、图算法
- 三、算法题1 (15分) } 小班讨论
四、算法题2 (15分) }

开卷考试, 限定带《算法导论》书
算法导论思考题答案: <https://walkccc.me/CLRS/>

目录

第一节

回溯与分支限界原理

第二节

批任务处理问题

第三节

0-1背包问题



□理论上

- 寻找问题的解的一种可靠的方法是首先列出所有候选解，然后依次检查每一个，在检查完所有或部分候选解后，即可找到所需要的解。

□但是

- 当候选解数量有限并且通过检查所有或部分候选解能够得到所需解时，上述方法是可行的。
- 若候选解的数量非常大（指数级，大数阶乘），即便采用最快的计算机也只能解决规模很小的问题。



□ 于是

- 回溯和分支限界法是比较常用的对候选解进行系统检查两种方法。
- 按照这两种方法对候选解进行系统检查通常会使问题的求解时间大大减少。
- 可以避免对很大的候选解集合进行检查，同时能够保证算法运行结束时可以找到所需要的解。
- 通常能够用来求解规模很大的问题。



□ 问题的解向量

回溯法与分支限界法把一个问题的解能够表示成一个 n 元组 $(x_1, x_2, \dots, x_n)$ 的形式。

针对输入 n 个元素，常用问题可抽象为一个 n 阶判断问题。



□ 两种常见的 n 阶判断问题解向量

1. 子集问题

目标：从 n 个元素中选出一个子集

决策过程：依次判断每个元素是否被选中

解向量形式： $(x_1, x_2, \dots, x_n)$ ，其中 $x_i \in \{0, 1\}$ (0 表示不选，1 表示选中)

2. 排列问题

目标：找出 n 个元素的一个排列

决策过程：依次确定排列中每个位置放置哪个元素

解向量形式： $(x_1, x_2, \dots, x_n)$ ，其中 x_i 是输入元素

解空间实例



对于有 n 种可选物品的0-1背包问题

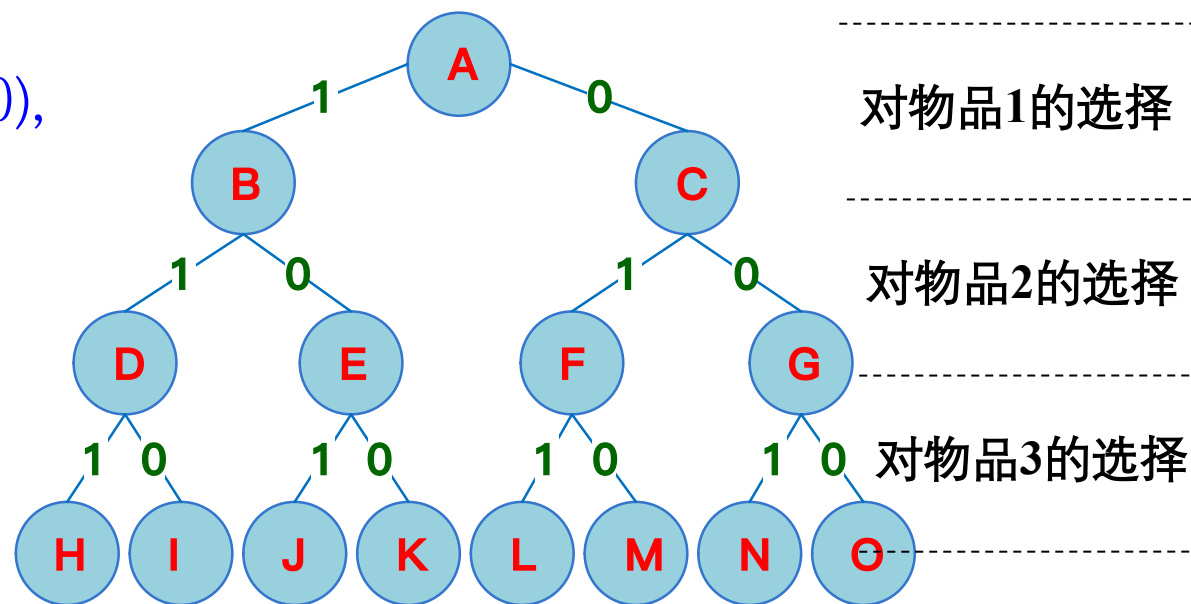
✓ 解空间由 2^n 个长度为 n 的0-1向量组成

✓ $n=3$ 时, 解空间为 $\{(0,0,0), (0,0,1), (0,1,0), (0,1,1), (1,0,0), (1,0,1), (1,1,0), (1,1,1)\}$

用**完全二叉树**表示的**解空间**

✓ 边上的数字给出了向量 x 中第 i 个分量的值 x_i

✓ **根节点到叶节点**的路径定义了解问题的一个解



典型的解空间树——子集树



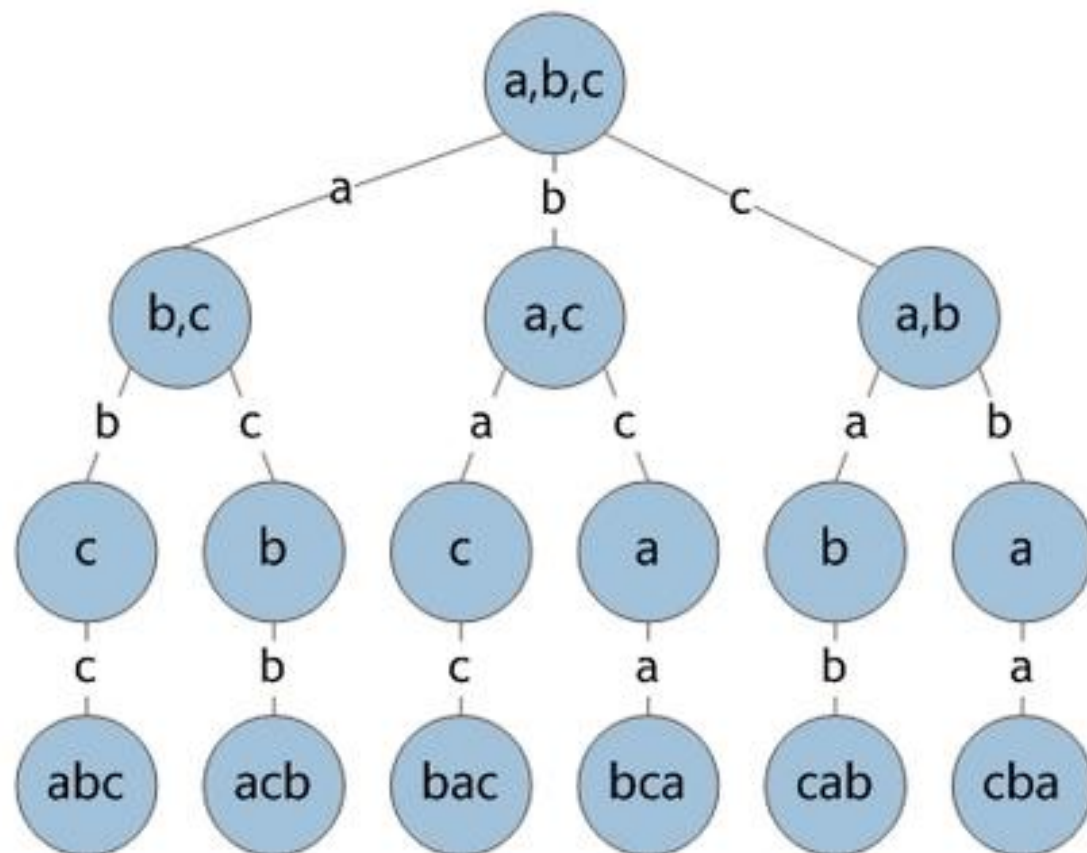
- 从 n 个元素的集合 S 中找出**S满足某种性质的子集**，相应的解空间称为子集树。通常有 2^n 个叶结点，结点总数为 $2^{n+1}-1$ 。需 $\Omega(2^n)$ 计算时间。
- 如 n 个物品的0-1背包问题的解空间是一棵子集树。

典型的解空间树——排列树



排列树 (Permutation Trees) : 当所给问题是确定 n 个元素满足某种性质的排列时, 相应的解空间树称为排列树。在排列树中, 通常情况下, $|S_0|=n$, $|S_1|=n-1$, ..., $|S_{n-1}|=1$, 所以, 排列树中共有 $n!$ 个叶子结点, 因此, 遍历排列树需要 $O(n!)$ 时间。

排列树实例





□ 显约束

对分量 x_i 的取值限定。

□ 隐约束

为满足问题的解而对不同分量之间施加的约束

解空间



□ 对于问题的一个实例，解向量满足显式约束条件的所有多元组，构成了该实例的一个解空间。

注意：同一问题可有多种表示，有些表示更简单，所需状态空间更小（存储量少，搜索方法简单）。

回溯法



- 回溯法实际上一个类似枚举的搜索尝试过程，主要是在搜索尝试过程中寻找问题的解，当发现已不满足求解条件时，就“回溯”返回，尝试别的路径。
- 回溯法以深度优先的方式系统地搜索问题的解，按择优条件向前搜索，以达到目标。

分支限界法的基本思想



- 分支限界法通常以**广度优先**或以**最小耗费（最大效益）** **优先**的方式搜索问题的解空间树。
- 在分支限界法中，每一个活结点**只有一次机会成为扩展结点**。活结点一旦成为扩展结点，就一次性产生其所有儿子结点。其中导致不可行解或非最优解的儿子结点被**舍弃**，其余儿子结点被**加入活结点表**中。
- 从活结点表中取下一结点成为当前扩展结点并重复上述结点扩展过程，直到找到所求的解或活结点表为空时为止。

目录

第一节

回溯与分支限界原理

第二节

批任务处理问题

第三节

0-1背包问题

基于回溯法的批处理作业调度



问题描述:

- 给定 n 个作业的集合 $\{J_1, J_2, \dots, J_n\}$ 。
- 每一个 J_i 作业都有两项任务分别在两台机器上完成。每个作业必须先由机器1处理，然后由机器2处理。
- 作业 J_i 需要机器 j 的处理时间为 t_{ji} ，其中 $i=1, 2, \dots, n$ ， $j=1, 2$ 。
- 对于一个确定的作业调度，设 F_{ji} 是作业 i 在机器 j 上完成处理的时间。
- 所有作业在**机器2**上完成处理的时间和 $f = \sum_{i=1}^n F_{2i}$ 称为该作业调度的完成时间和。

基于回溯法的批处理作业调度



批处理作业调度问题要求对于给定的 n 个作业，制定最佳作业调度方案，使其完成时间和达到最小

批处理作业调度实例



问题解析:

t_{ji}	机器1	机器2
作业1	2	3
作业2	5	2
作业3	4	1

➤ 这个3个作业的6种可能的调度方案是:

(1,2,3) (1,3,2) (2,1,3)

(2,3,1) (3,1,2) (3,2,1)

➤ 对应的完成时间和分别是: 12, 13, 12, 14, 13, 16。

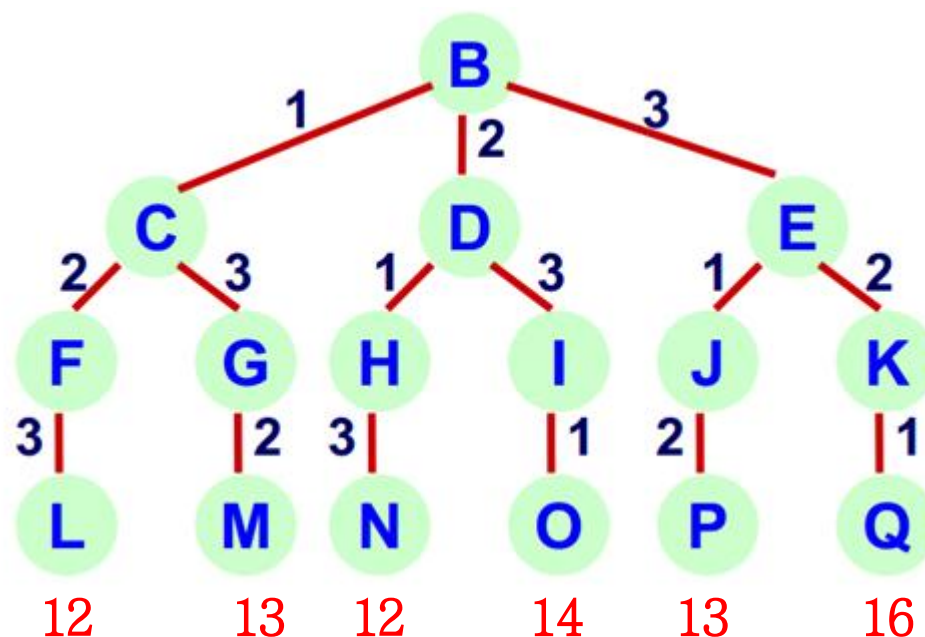
➤ 最佳调度方案是1,2,3和2,1,3, 其完成时间和为12。

批处理作业调度



算法设计:

批处理作业调度问题要从 n 个作业的所有排列中找出有最小完成时间和的作业调度，所以批处理作业调度问题的解空间是一颗**排列树**。



基于分支限界的批处理作业调度问题



问题描述：给定 n 个作业的集合 $J=\{J_1, J_2, \dots, J_n\}$ ，每个作业都有3项任务分别在3台机器上完成，作业 J_i 需要机器 j 的处理时间为 $t_{ij}(1 \leq i \leq n, 1 \leq j \leq 3)$ ，每个作业必须先由机器1处理，再由机器2处理，最后由机器3处理。

- 批处理作业调度问题要求确定这 n 个作业的最优处理顺序，使得从第1个作业在机器1上处理开始，到最后一个作业在机器3上处理结束所需的时间最少。

批处理作业调度问题实例



实例：设 $J=\{J_1, J_2, J_3, J_4\}$ 是4个待处理的作业，需要的处理时间如下所示

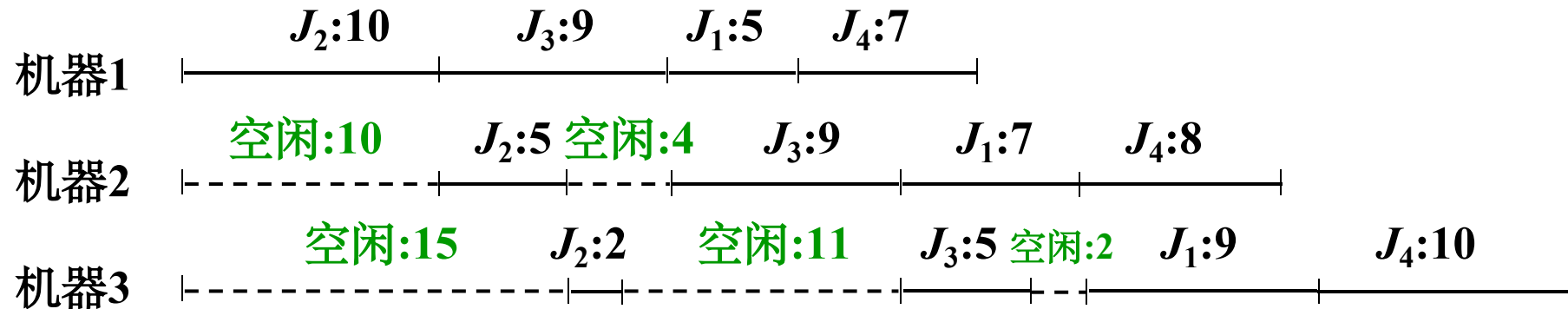
	机器1	机器2	机器3
J_1	5	7	9
J_2	10	5	2
J_3	9	9	5
J_4	7	8	10

若处理顺序为 (J_2, J_3, J_1, J_4) ，则从作业2在机器1处理开始到作业4在机器3处理完成的调度方案如下：

批处理作业调度问题实例



$$T = \begin{matrix} & \begin{matrix} \text{机器1} & \text{机器2} & \text{机器3} \end{matrix} \\ \begin{matrix} J_1 \\ J_2 \\ J_3 \\ J_4 \end{matrix} & \begin{bmatrix} 5 & 7 & 9 \\ 10 & 5 & 2 \\ 9 & 9 & 5 \\ 7 & 8 & 10 \end{bmatrix} \end{matrix}$$



(----表示空闲, 最后完成处理时间为54)

↓
等待时间+处理时间

批处理作业调度问题实例



分析:

- 解向量: $X=(x_1, x_2, \dots, x_n)$ ——排列树
- 约束条件: 显式: $x_i=J_1, J_2, \dots, J_n$
- 目标函数: $sum3[n] = \max\{sum2[n], sum3[n-1]\} + t_{n3}$ 极小化

其中, $sum2[n] = \max\{sum1[n], sum2[n-1]\} + t_{n2}$;

$$sum1[n] = sum1[n-1] + t_{n1}$$

批处理作业调度问题实例



下限界函数： 设 M 是已安排好的作业集合， $M \subset \{1, 2, \dots, n\}$ ， $|M|=k$ 。现在要处理作业 $k+1$ ，有：

$$db = \max\{ \text{sum1}[k+1], \text{sum2}[k] \} + \sum_{i \notin M} t_{i2} + \min\{ t_{j3} \}_{j \neq k+1, j \notin M}$$

$$\text{sum1}[k+1] = \text{sum1}[k] + t_{k+1,1}$$

$$\text{sum2}[k+1] = \max\{\text{sum1}[k+1], \text{sum2}[k]\} + t_{k+1,2}$$

批处理作业调度问题实例



目标函数的下界: $sum3_{db} = t_{i1} + \sum_{j=1}^n t_{j2} + \min_{k \neq i} \{t_{k3}\}$

说明：最理想的情况下，机器1和机器2均无空闲，最后处理的作业恰好是在机器3上处理时间最短的作业。

如下实例， $sum3_{db} = t_{11} + \sum_{j=1}^4 t_{j2} + t_{23} = 36$

	机器1	机器2	机器3
J_1	5	7	9
J_2	10	5	2
J_3	9	9	5
J_4	7	8	10

批处理作业调度问题实例



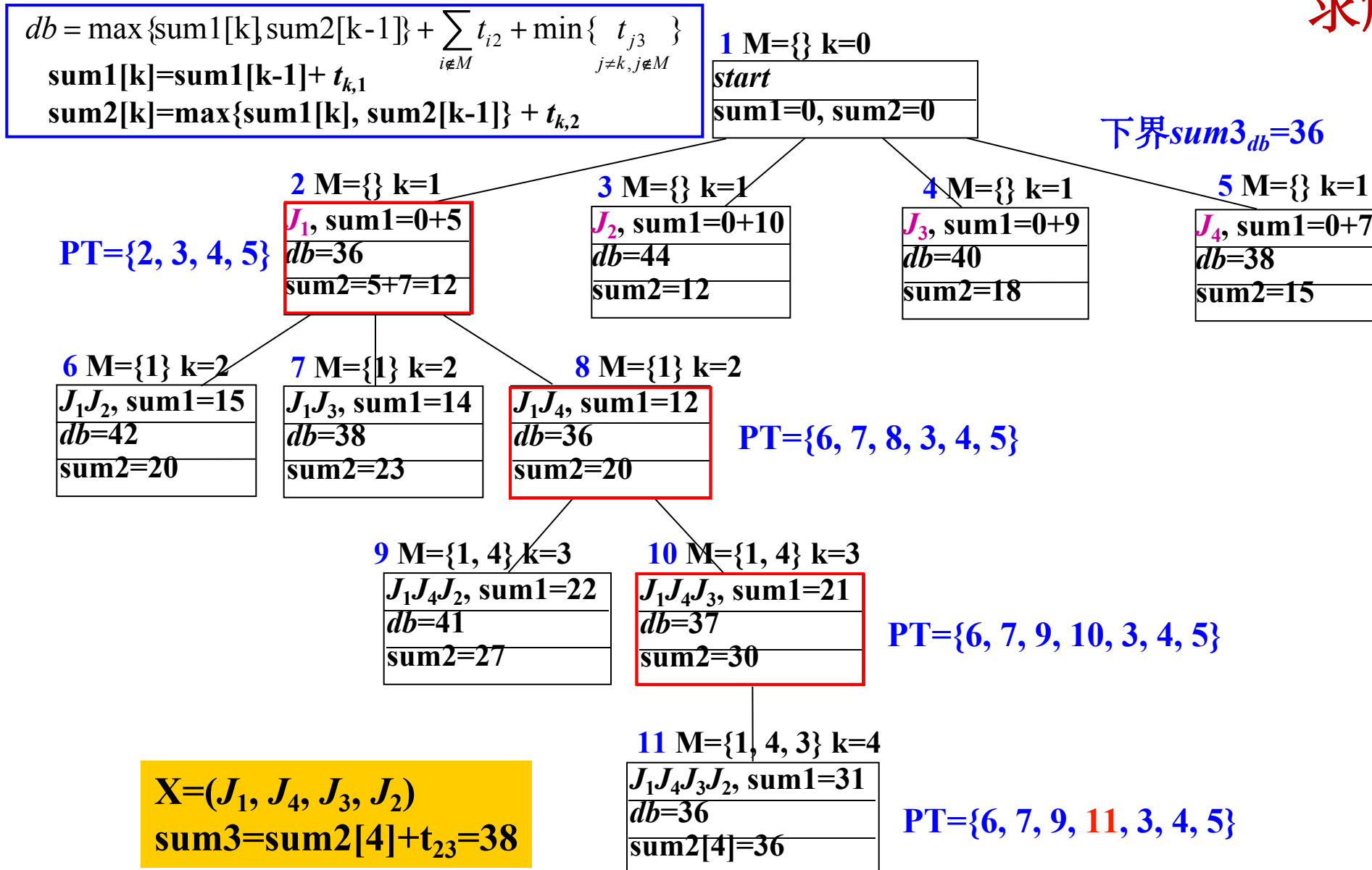
遍历并估算解空间树上各结点的目标函数的下界值:

根结点: $sum1=0, sum2=0, M=\{\}$

$k=0$

	机器1	机器2	机器3	
$T =$	J_1	5	7	9
	J_2	10	5	2
	J_3	9	9	5
	J_4	7	8	10

求解过程



批处理作业调度问题实例总结



从上例可知：分支限界法中，

- 扩展结点表PT取得极值的叶子结点就对应的是问题最优解；
- 扩展结点的过程，一开始实际类似“深度优先”

目录

第一节

回溯与分支限界原理

第二节

批任务处理问题

第三节

0-1背包问题

0-1背包问题



在0-1背包问题中，需对容量为 c 的背包进行装载。从 n 个物品中选取装入背包的物品，每件物品 i 的重量为 w_i ，价值为 v_i 。对于可行的背包装载，背包中的物品的总重量不能超过背包的容量，最佳装载是指所装入的物品价值最高

$$\max \sum_{1 \leq i \leq n} v_i x_i$$

$$\text{约束条件为: } \sum_{1 \leq i \leq n} w_i x_i \leq C, x_i \in \{0, 1\}$$

在这个表达式中，需求出 x_i 的值。 $x_i=1$ 表示物品 i 装入背包中， $x_i=0$ 表示物品 i 不装入背包。其中 C 、 w_i 以及 v_i 不限定为一个整数

0-1背包问题



- 令 $cw(i)$ 表示目前搜索到第 i 层已经装入背包的物品总重量，即部分解 $(x_1, x_2, \dots, x_i)$ 的重量：

$$cw(i) = \sum_{j=1}^i x_j w_j$$

- 对于左子树， $x_i=1$ ，其约束函数为：

$$constraint(i) = cw(i-1) + w_i$$

- 若 $constraint(i) > C$ ，则停止搜索左子树，否则继续搜索。

0-1背包问题



- 对于右子树，为了提高搜索效率，采用上界函数 $\text{Bound}(i)$ 剪枝。
- 令 $\text{cv}(i)$ 表示目前到第 i 层结点已经装入背包的物品价值：

$$\text{cv}(i) = \sum_{j=1}^i x_j v_j$$

- 令 $r(i)$ 表示剩余物品的总价值：

$$r(i) = \sum_{j=i+1}^n v_j$$

- 则限界函数 $\text{Bound}(i)$ 为：

$$\text{Bound}(i) = \text{cv}(i) + r(i)$$

0-1背包问题

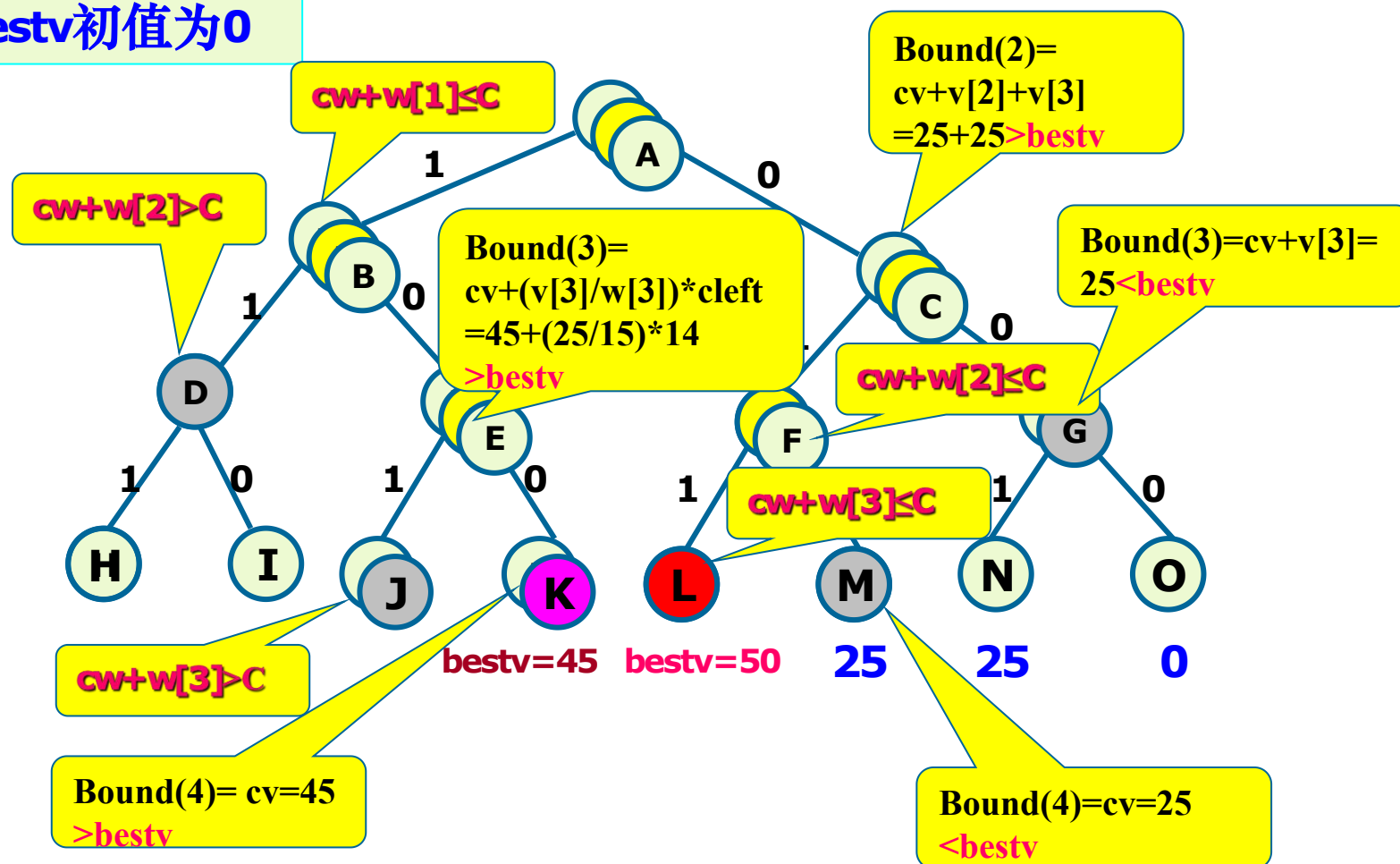


- 假设当前最优值为 $bestv$ ，若 $Bound(i) < bestv$ ，则停止搜索第 i 层结点及其子树，否则继续搜索。显然 $r(i)$ 越小， $Bound(i)$ 越小，剪掉的分支就越多。
- 为了构造更小的 $r(i)$ ，将物品以单位重量价值比 $d_i = v_i / w_i$ 递减的顺序进行排列： $d_1 \geq d_2 \geq \dots \geq d_n$
- 对于第 i 层，背包的剩余容量为 $C - cw(i)$ ，采用贪心算法把剩余的物品放进背包，根据贪心算法理论，此时剩余物品的价值已经是最优的，因为对剩余物品不存在比上述贪心装载方案更优的方案。

0-1背包问题



bestv初值为0



$n=3$, $W=\{16, 15, 15\}$,
 $v=\{45, 25, 25\}$, $C=30$.

0-1背包问题



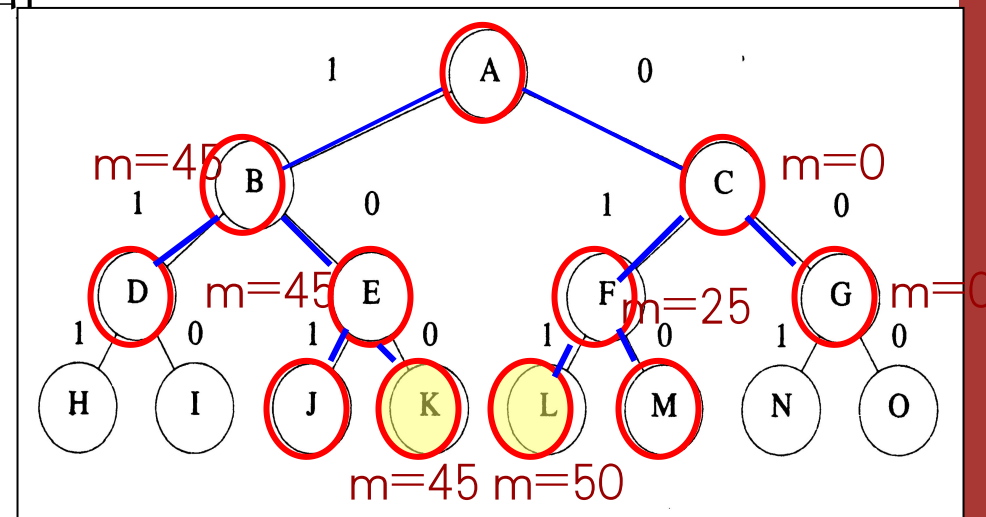
•算法效率

计算上界函数Bound 需要 $O(n)$ 时间，在最坏情况下有 $O(2^n)$ 个右儿子结点需要计算上界函数，所以解0-1背包问题的回溯算法

Backtrack的计算时间为：

$$T(n)=O(n2^n)$$

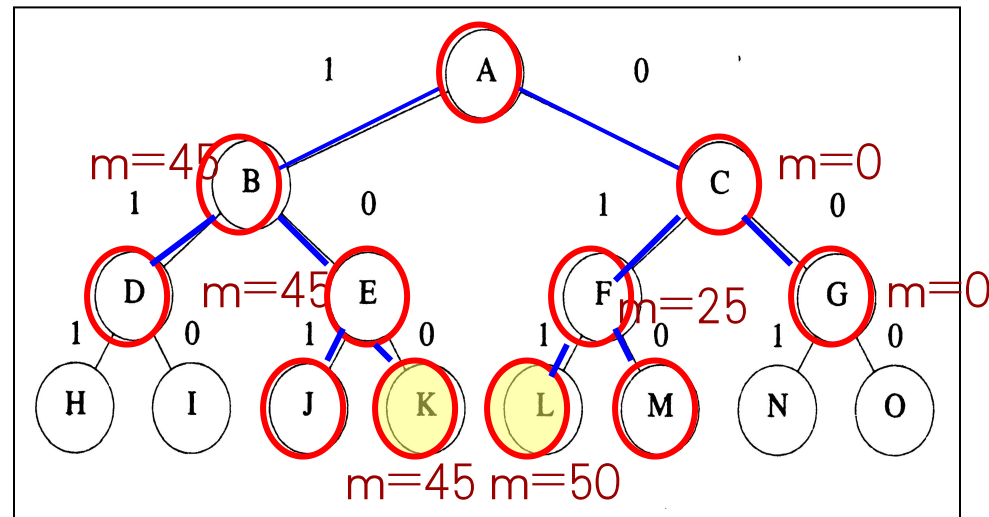
叶结点, 是问题的一个可行解, 价值为45



0-1背包问题-队列式分支限界法



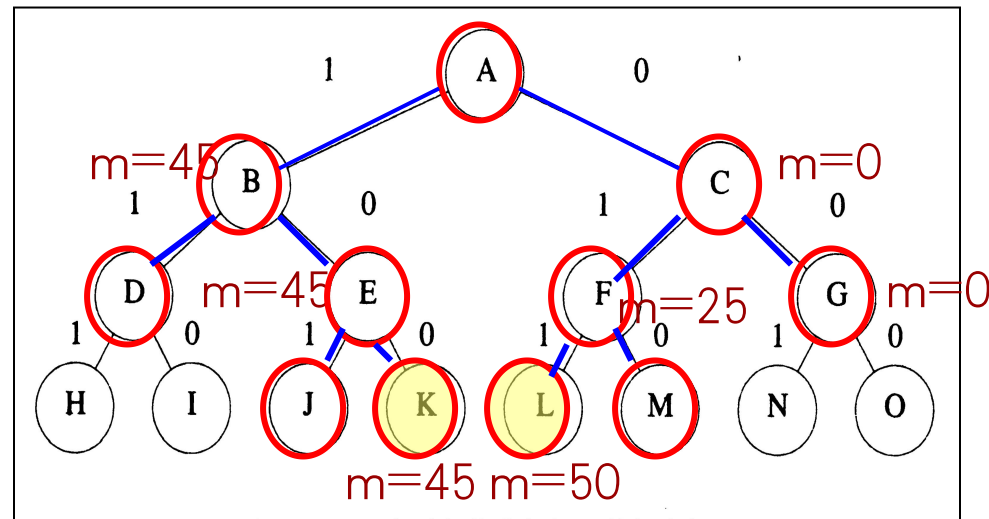
- 当前活结点队列的队首为F, 儿子结点L、M为可行叶结点, 价值为50、25
- G为最后一个扩展结点, 儿子结点N、O均为可行叶结点, 其价值为25和0
- 活结点队列为空, 算法结束, 其最优值为50



0-1背包问题-队列式分支限界法



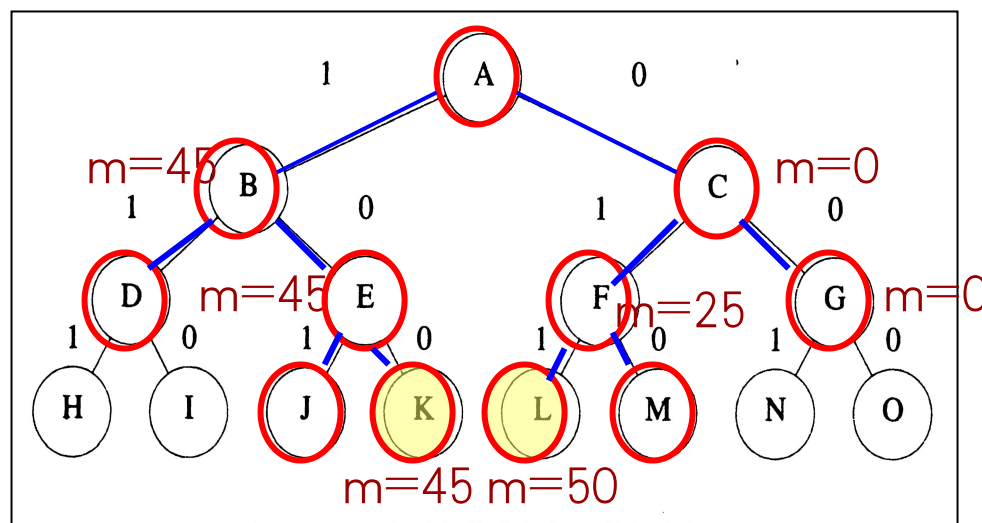
- 用一个极大堆表示活结点表的优先队列，其优先级定义为活结点所获得的价值。初始为空。
- 由A开始搜索解空间树，其儿子结点B、C为可行结点，加入堆中，舍弃A。
- B获得价值45，C为0. B为堆中价值最大元素，并成为下一扩展结点。



0-1背包问题-队列式分支限界法



- B的儿子结点D是不可行结点，舍弃。E是可行结点，加入到堆中。舍弃B。
- E的价值为45，是堆中最大元素，为当前扩展结点。
- E的儿子J是不可行叶结点，舍弃。K是可行叶结点，为问题的一个可行解价值为45。
- 继续扩展堆中唯一活结点C，直至存储活结点的堆为空，算法结束。
- 算法搜索得到最优值为50，最优解为从根结点A到叶结点L的路径 (0, 1, 1) 。





湖南大学
HUNAN UNIVERSITY

谢谢观赏

—— 实事求是 敢为人先 ——